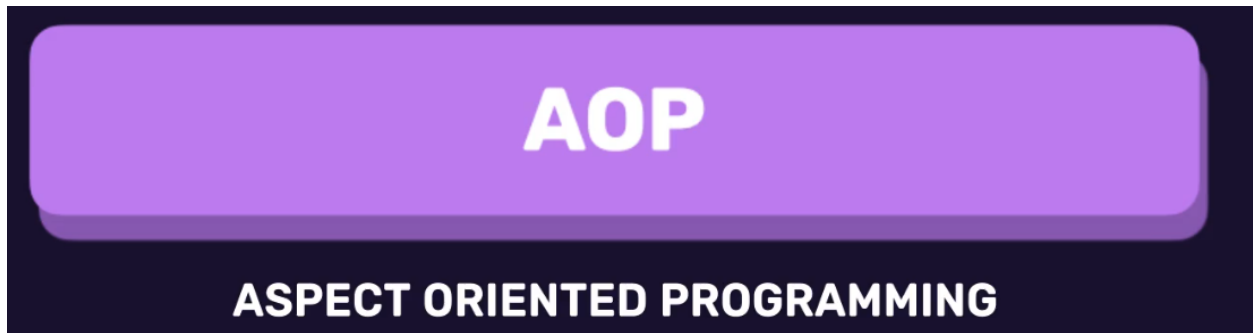
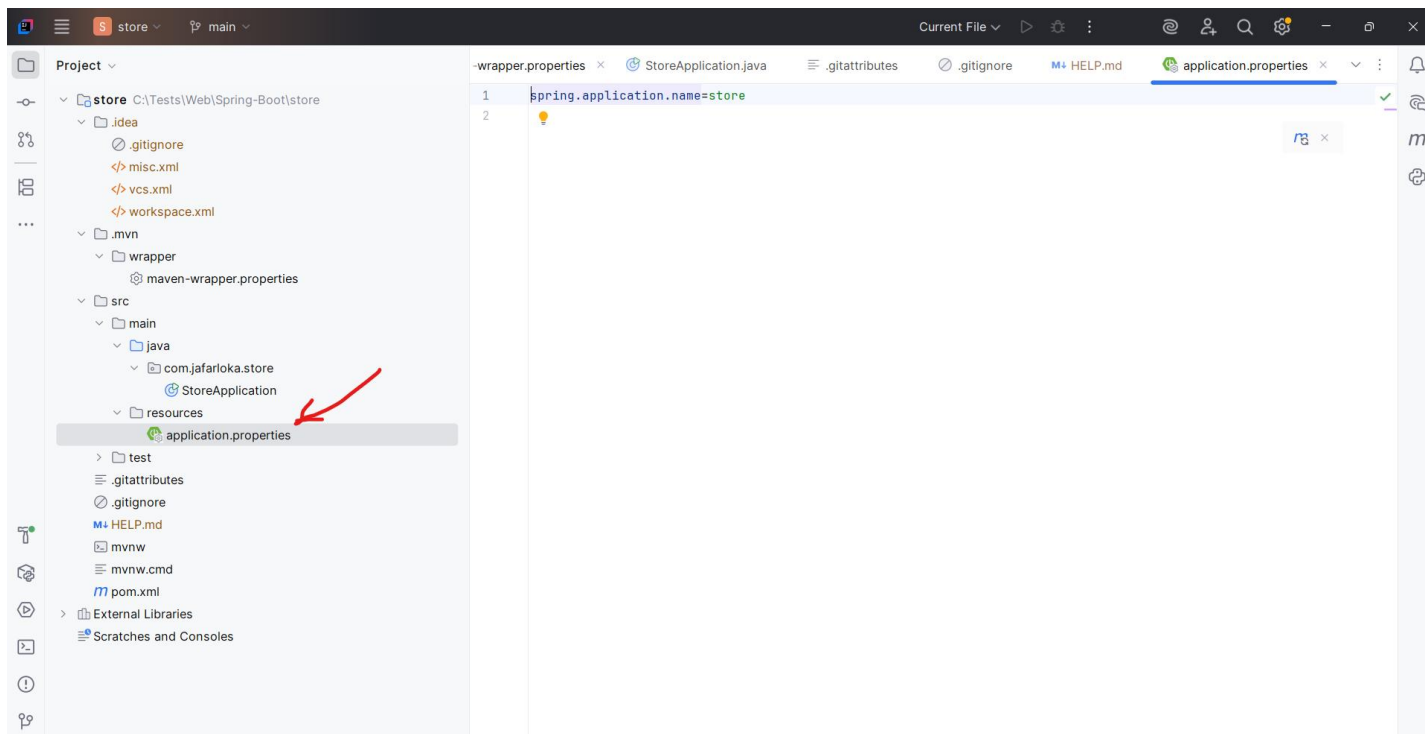
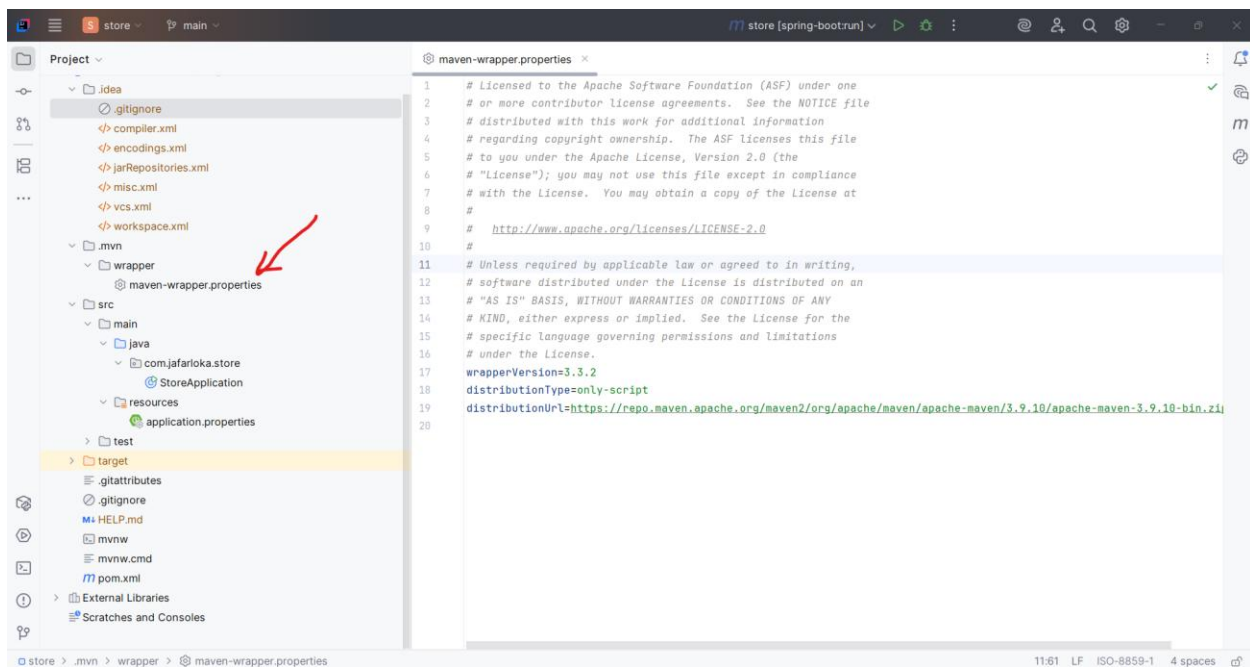


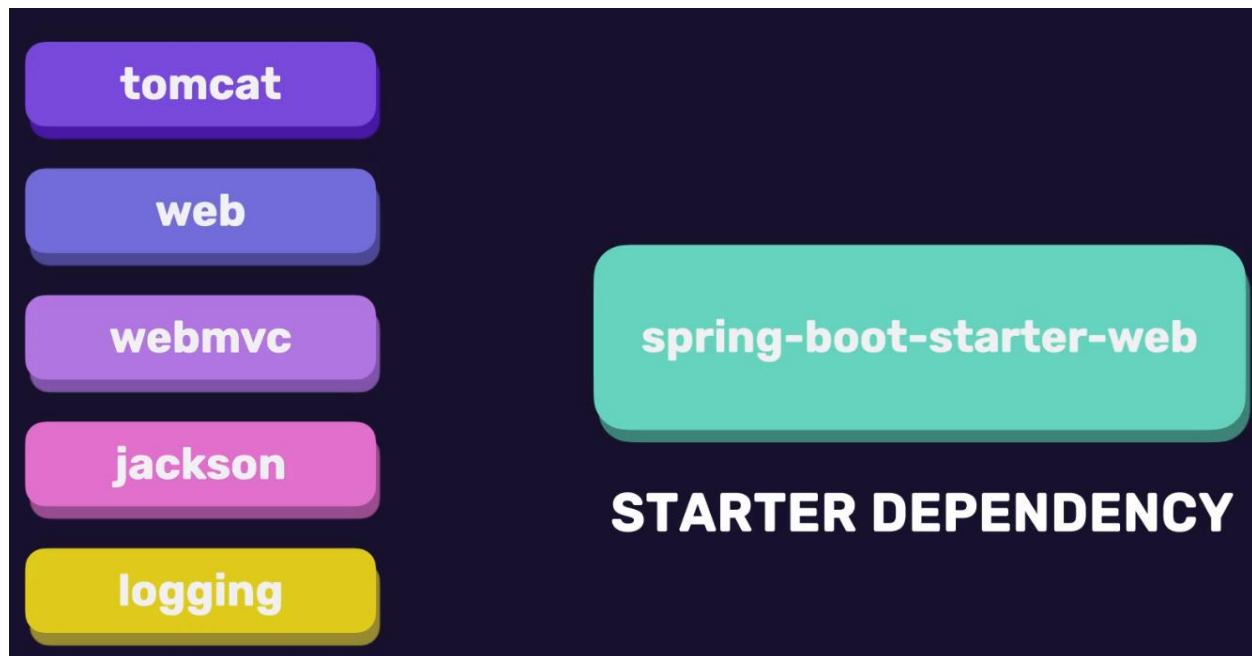
AOP used for adding features that doesn't exists in framework (to extend it).







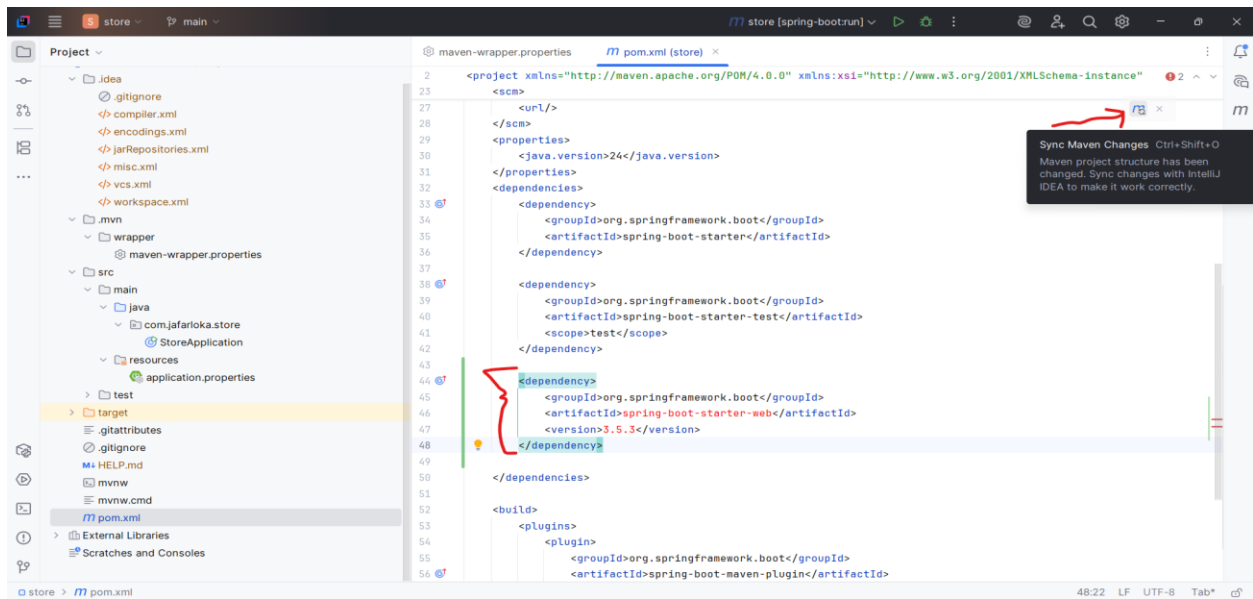


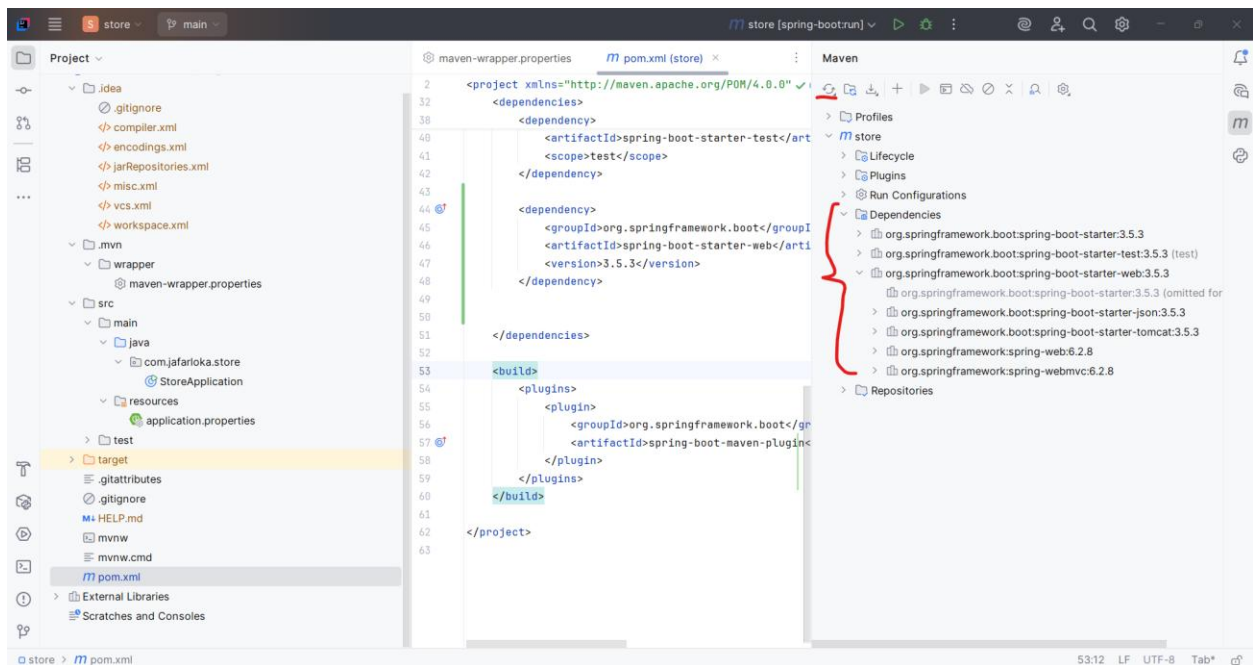


Maven Central: <https://central.sonatype.com/>

Here we can search for dependencies like **spring-boot-starter-web**

After adding the dependency, may it not found in local machine, so we must download it:





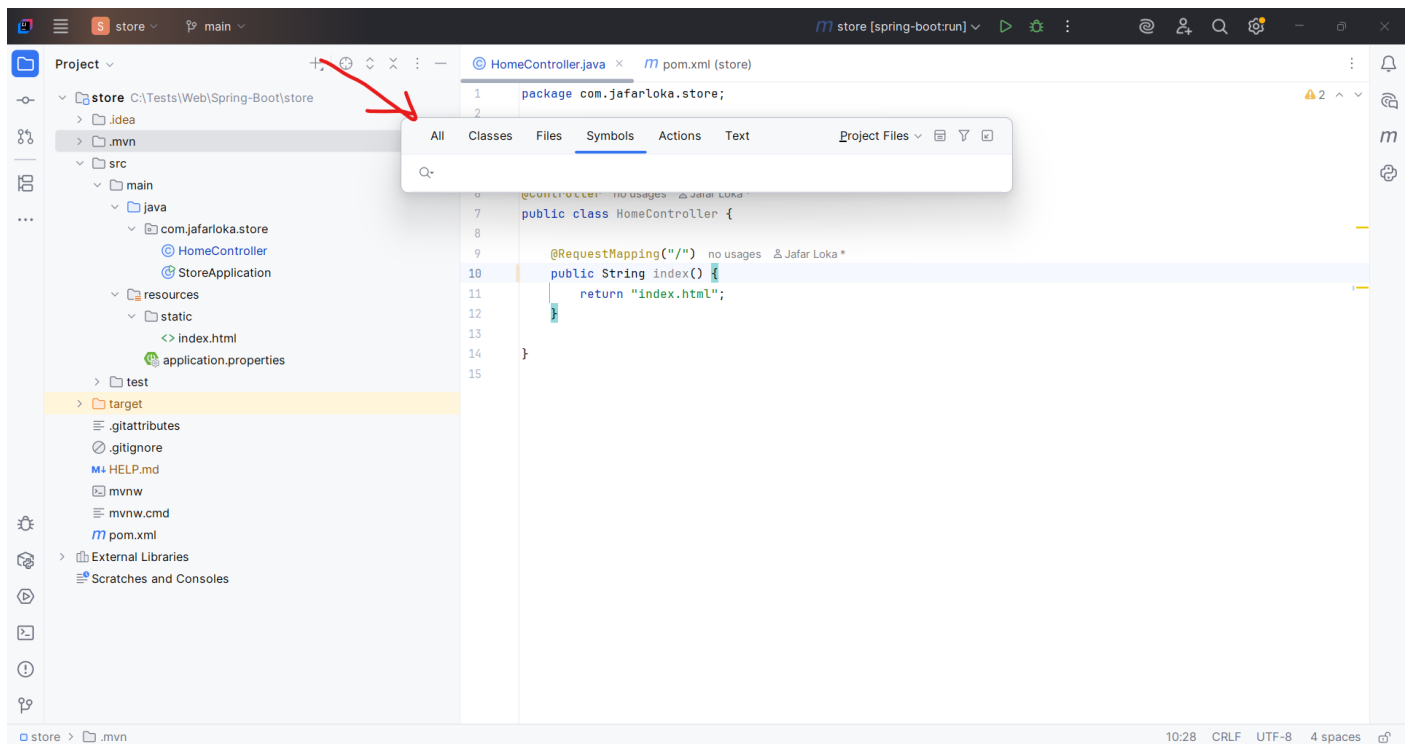
Always we can delete the version-tag in our dependency in pom.xml file, and then we make the maven and our project determine the best version for our project

We can run spring boot project using: *mvnw spring-boot:run*

OR from:



SHIFT + CTRL + O → For searching for file in IntelliJ IDE



ALT + 0 → Generate dependency

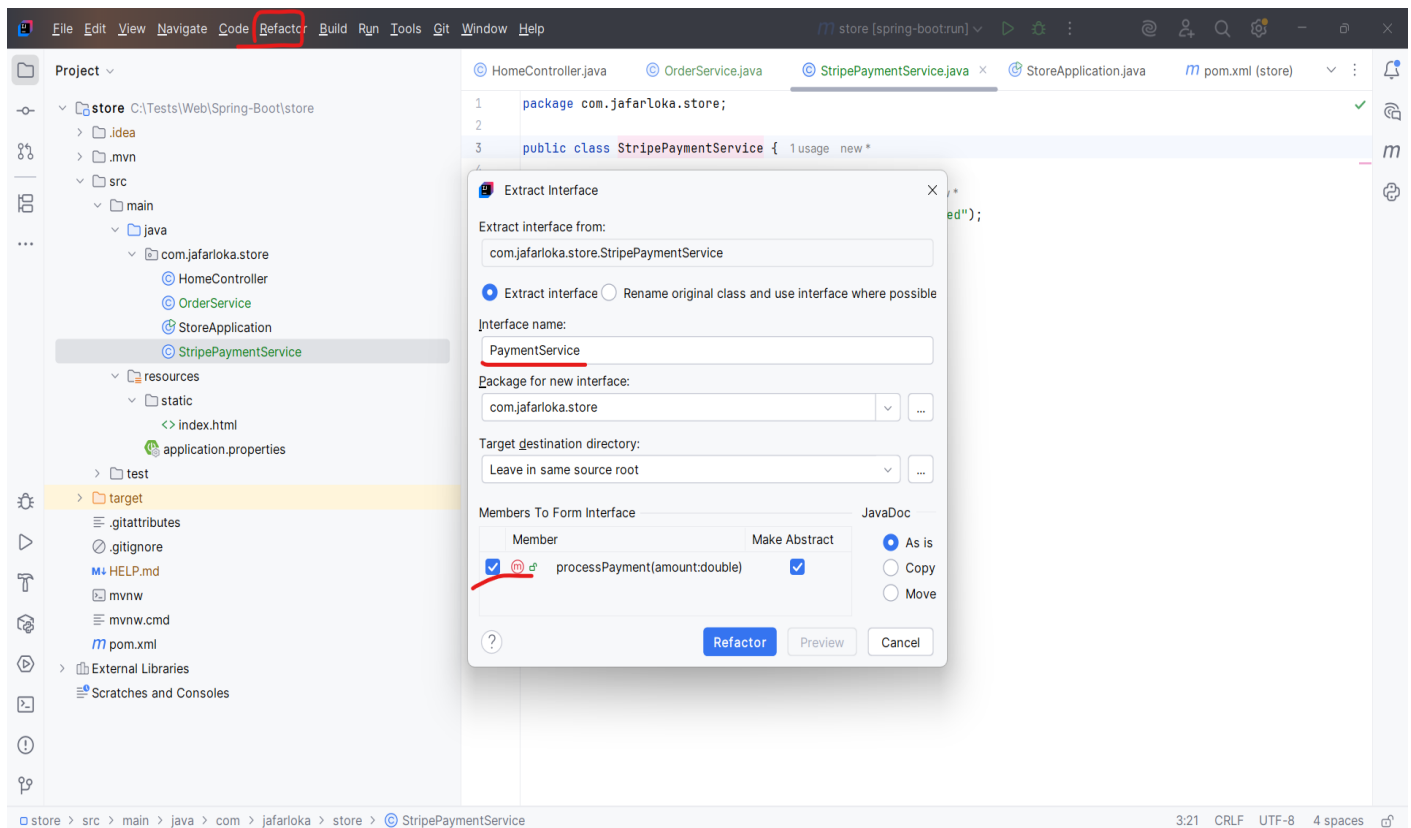
To define specific configuration, we go to application.properties:

```
spring.application.name=store
jloka.value=100
```

Then to use their value on code, we use Value-annotation, with \${}:

```
@Value("${spring.application.name}")
private String appName;
```

```
@Value("${jloka.value}")
private String value;
```



Note: tight couples mean we have bad service coupling with other, may it can be in small projects, or in projects that have less features in some part of them.

From solid principle, And OOP:

OPEN CLOSED PRINCIPLE

A class should be
open for extension and
closed for modification.

We can use DI using setter-methods:

```
public class OrderService {  
  
    private PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
    public void setPaymentService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
    public void placeOrder() {  
        paymentService.processPayment(100_000);  
    }  
}
```

We can use Generate from IntelliJ to create the setter methods, or other tasks:

```
public class OrderService { 3 usages  Jafar Loka *

    private PaymentService paymentService; 3 usages

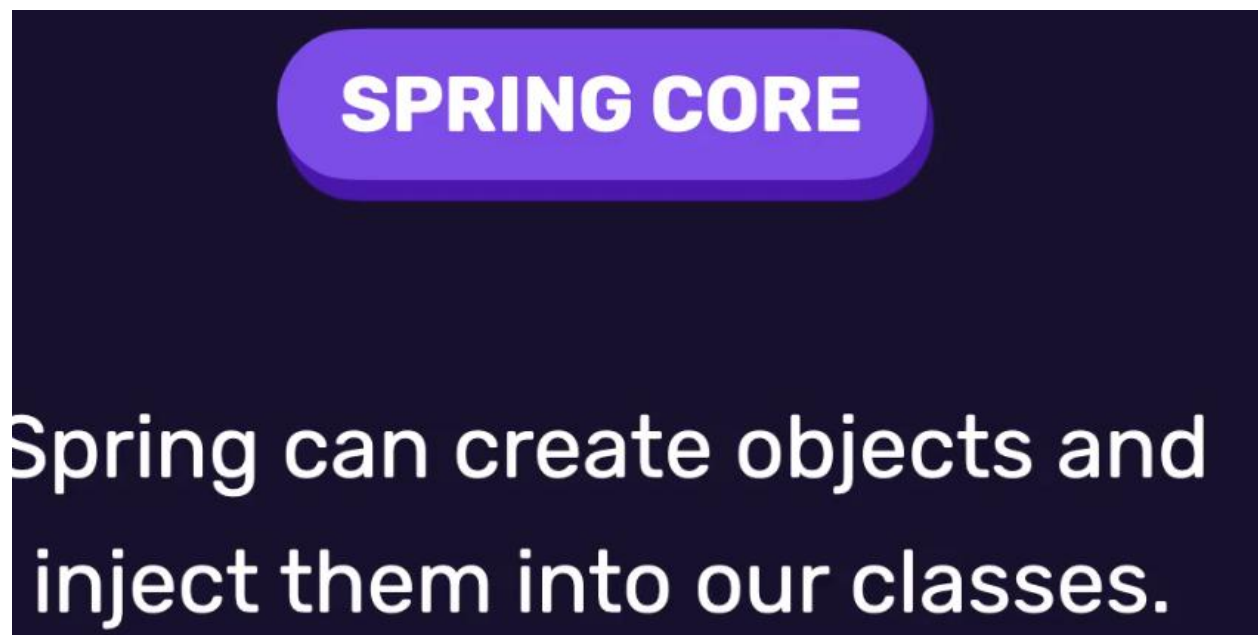
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Generate

- Constructor
- Logger
- Getter
- Setter
- Getter and Setter
- equals() and hashCode()
- toString()
- Override Methods...
- Delegate Methods...
- Test...
- Copyright

```
4:06:12.270+03:00 INFO 13372 --- [store] o.a.c.c.C.[Tomcat].[/]
4:06:12.313+03:00 INFO 13372 --- [store] s.c.ServletWebServerApplicationContext
4:06:12.314+03:00 INFO 13372 --- [store] s.b.a.w.s.WelcomePageHandlerMapping
4:06:12.371+03:00 INFO 13372 --- [store] s.b.w.embedded.tomcat.TomcatWebServer
4:06:12.533+03:00 INFO 13372 --- [store] m.jafarloka.store.StoreApplication
4:06:12.537+03:00 INFO 13372 --- [store] m.jafarloka.store.StoreApplication
4:06:16.342+03:00 INFO 13372 --- [store] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[/]
*****
```



IOC Container: set of objects that can be used for DI, each object called **BEAN**

Bean: Regular java code that used by spring core.

IOC: Inversion Of Control → Invert the control of *creating objects*, and *managing dependencies*.



To make IOC control the DI of OrderService class:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class StoreApplication {

    public static void main(String[] args) {
        ApplicationContext context =
        SpringApplication.run(StoreApplication.class, args);

        context.getBean(OrderService.class);
    }
}
```

To manage beans inside code:

CONFIGURING BEANS

- Using annotations
- Using code